Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Sistemas de Bases de Datos 2

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión de Reservas y Boletos Aéreos con Apache Cassandra

Proyecto 1 — Segundo semestre 2026

Victor Ronaldo Gómez Lara
Carné: 201114493

Guatemala, septiembre de 2026

Índice

1. Introducción

Este documento describe el diseño, la implementación y la validación de una base de datos distribuida en Apache Cassandra 4.1 para la gestión de reservas y venta de boletos de una aerolínea. El sistema administra seis entidades del negocio —pasajeros, aeronaves, vuelos, asientos, reservas y pagos— y fue construido siguiendo el principio de modelado dirigido por consultas (Query-Driven Modeling), en el cual las tablas se derivan de las preguntas que el sistema debe responder y no de las entidades del dominio.

A diferencia de un motor relacional, CQL no dispone de la operación JOIN ni de integridad referencial. La resolución de una consulta que involucra varias entidades se traslada al momento de la escritura mediante denormalización controlada: los datos se copian en tantas tablas como patrones de acceso existan, de modo que cada lectura se resuelva en una sola partición. El resultado es un esquema de quince tablas físicas que da servicio a las cinco consultas exigidas por el negocio sin recurrir en ningún caso a ALLOW FILTERING.

El clúster se desplegó sobre Docker con tres nodos y factor de replicación 3. Se cargaron 100,080 reservas mediante Batch Writes y se midió el comportamiento del sistema ante la caída de un nodo bajo los niveles de consistencia ONE, QUORUM y ALL.

2. Arquitectura y configuración del clúster

2.1 Topología

El clúster está formado por tres nodos de Apache Cassandra 4.1.12 desplegados como contenedores Docker sobre una red bridge dedicada. Los tres nodos pertenecen al datacenter lógico dc1 y al rack rack1. El nodo cass1 actúa como semilla (seed) y es el único que expone el puerto 9042 hacia el host, de modo que los clientes externos —cqlsh y el driver de Python— se conectan a él y el propio protocolo se encarga de descubrir el resto de la topología.

Parámetro	Valor	Justificación
Nombre del clúster	aerolinea_cluster	Identificador común a los tres nodos; un nombre distinto impediría la formación del anillo.
Versión	Apache Cassandra 4.1.12	Cumple el requisito de versión 4.x o superior.
Nodos	3 (cass1, cass2, cass3)	Mínimo exigido para evaluar QUORUM de forma significativa.
Snitch	GossipingPropertyFileSnitch	Requerido por NetworkTopologyStrategy; permite declarar datacenter y rack por nodo.
Particionador	Murmur3Partitioner	Predeterminado; distribuye uniformemente las claves de partición por hash.
Tokens por nodo	16 (vnodes)	Distribución equilibrada del anillo sin cálculo manual de tokens.
Heap JVM	512 MB por nodo	Ajustado al entorno de pruebas local de 8 GB de memoria física.

El arranque de los nodos es secuencial y está controlado mediante healthchecks en docker-compose: cass2 espera a que cass1 responda a cqlsh, y cass3 espera a cass2. Un arranque simultáneo provoca que cada nodo forme su propio anillo y el clúster quede particionado.

```
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2526_Proyectos_201114493/Primer Proyecto$ docker compose ps
NAME      IMAGE          COMMAND                  SERVICE   CREATED       STATUS                  PORTS
cass1     cassandra:4.1  "docker-entrypoint.s…"   cass1     6 hours ago   Up 6 hours (healthy)    7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::
]:9042->9042/tcp
cass2     cassandra:4.1  "docker-entrypoint.s…"   cass2     6 hours ago   Up 6 hours (healthy)    7000-7001/tcp, 7199/tcp, 9042/tcp, 9160/tcp
cass3     cassandra:4.1  "docker-entrypoint.s…"   cass3     6 hours ago   Up 10 minutes           7000-7001/tcp, 7199/tcp, 9042/tcp, 9160/tcp
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2526_Proyectos_201114493/Primer Proyecto$ docker exec cass1 nodetool status
Datacenter: dc1
===============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Host ID                               Rack
UN  172.19.0.2  37.63 MiB  16      100.0%            5af76af3-732e-4761-8ef4-8100a3b599bf  rack1
UN  172.19.0.4  37.61 MiB  16      100.0%            4ce13d2a-99f8-4b45-b0bc-262bee75ff3b  rack1
UN  172.19.0.3  37.7 MiB   16      100.0%            4873727c-c1bc-48d3-8980-b3fa22a64e75  rack1
```

Figura 1. Contenedores activos y estado del anillo (nodetool status).

2.2 Estrategia de replicación y factor de replicación

El keyspace se creó con NetworkTopologyStrategy y factor de replicación 3 sobre dc1:

```
CREATE KEYSPACE aerolinea
WITH replication = {
  'class': 'NetworkTopologyStrategy',
  'dc1': 3
};
```

La elección de NetworkTopologyStrategy sobre SimpleStrategy responde a dos motivos. El primero es de portabilidad: SimpleStrategy ubica las réplicas recorriendo el anillo sin considerar la topología física, por lo que puede colocar las tres copias en el mismo rack o datacenter; NetworkTopologyStrategy distribuye las réplicas entre racks distintos y permite declarar un factor de replicación independiente por datacenter. El segundo es de evolución: el factor de replicación de un keyspace creado con SimpleStrategy no puede extenderse a un segundo datacenter sin migrar la estrategia, operación que exige reparación completa de los datos. Aunque el despliegue actual es de un solo datacenter, la documentación oficial de Apache Cassandra desaconseja SimpleStrategy para cualquier entorno distinto de pruebas desechables.

El factor de replicación 3 sobre tres nodos implica que cada nodo mantiene una copia completa de los datos —lo que se refleja en el valor Owns (effective) de 100.0% para los tres nodos— y hace que QUORUM equivalga a dos réplicas. Esa combinación es la que permite tolerar la caída de un nodo conservando consistencia fuerte, como se demuestra en la sección 8.

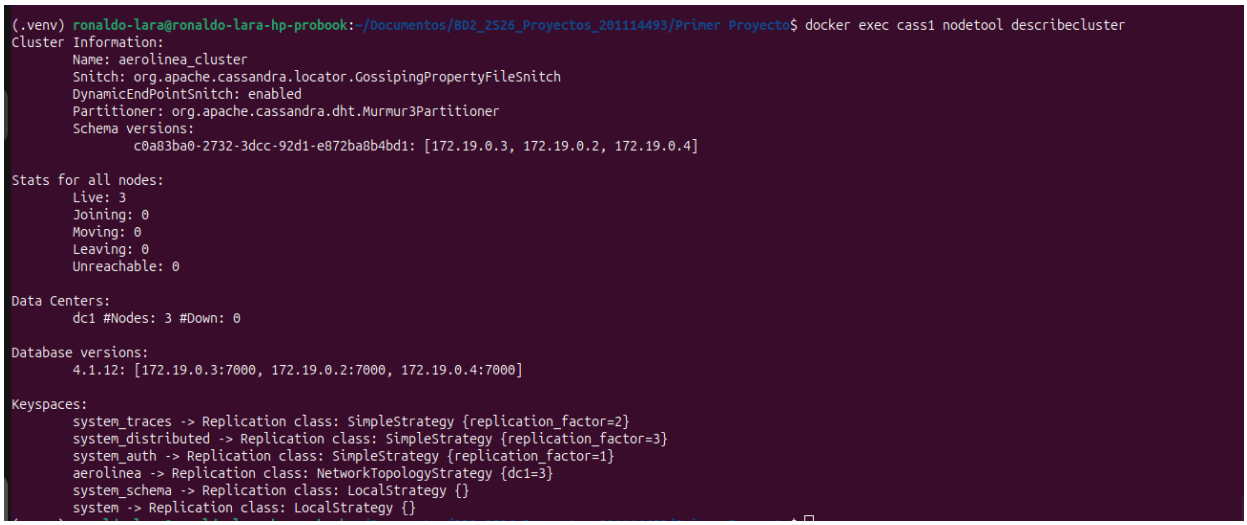


Figura 2. Información del clúster: versión de esquema única, tres nodos vivos y estrategia de replicación del keyspace.

2.3 Nivel de consistencia

Las operaciones de la aplicación se ejecutan con QUORUM. Con factor de replicación 3, QUORUM exige respuesta de dos réplicas tanto en lectura como en escritura, de manera que se cumple la desigualdad R + W > RF (2 + 2 > 3). Esa condición garantiza que todo conjunto de lectura y todo conjunto de escritura comparten al menos una réplica, por lo que una lectura nunca devuelve un valor anterior a la última escritura confirmada. El nivel ALL ofrece la misma garantía de consistencia pero sin tolerancia a fallos, y el nivel ONE ofrece la latencia más baja a costa de poder devolver datos desactualizados.

3. Modelo conceptual entidad-relación

El modelo conceptual representa la lógica del negocio con independencia del motor de base de datos. Se identificaron seis entidades y las siguientes relaciones:

- Una aeronave opera muchos vuelos; cada vuelo es operado por una sola aeronave (1:N).
- Un vuelo contiene muchos asientos; cada asiento pertenece a un único vuelo (1:N, dependencia de existencia).
- Un pasajero genera muchas reservas; cada reserva corresponde a un solo pasajero (1:N).
- Un vuelo recibe muchas reservas; cada reserva corresponde a un solo vuelo (1:N).
- Un asiento de un vuelo específico se asigna a una sola reserva activa a la vez (1:1 restringido).
- Cada reserva tiene asociado exactamente un pago (1:1).

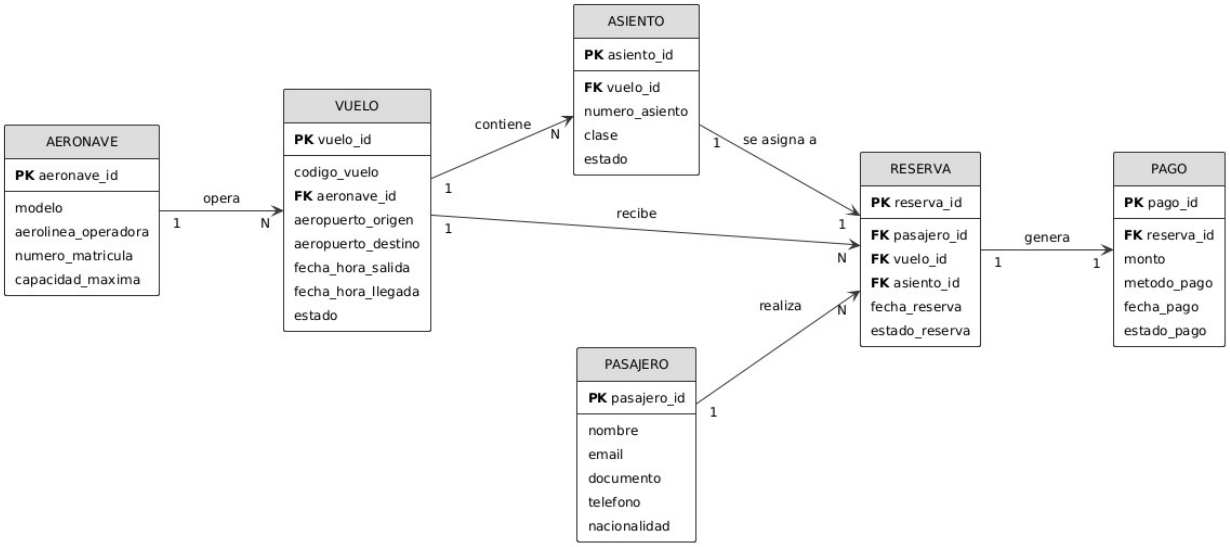


Figura 3. Modelo entidad-relación conceptual del sistema de reservas aéreas.

Las reglas de negocio que condicionan el diseño son las siguientes: un asiento de un vuelo solo puede estar asignado a un pasajero a la vez; el número de reservas confirmadas no puede superar la capacidad de la aeronave asignada; las reservas admiten modificación o cancelación hasta veinticuatro horas antes de la salida; la cancelación de un vuelo debe propagarse a sus reservas y pagos; y el sistema debe responder consultas agregadas de ocupación e ingresos, no solo listados.

4. Modelo lógico basado en queries

4.1 Principio de diseño

En Cassandra la unidad de acceso es la partición. La clave de partición determina, mediante el hash Murmur3, el nodo en el que reside la fila; una consulta que la proporciona se resuelve contactando un único nodo, mientras que una consulta que no la proporciona obliga a interrogar a todo el anillo y filtrar en memoria, lo que exige la cláusula ALLOW FILTERING y degrada el rendimiento de forma proporcional al tamaño del conjunto de datos.

Por ese motivo el diseño parte de las consultas. Se enumeraron los cinco patrones de acceso requeridos por el negocio y se definió una tabla por patrón, replicando en cada una los atributos necesarios para que la respuesta se obtenga sin acceder a otra tabla. Las seis tablas restantes corresponden a las entidades del modelo conceptual y sirven para la consulta puntual por identificador.

4.2 Correspondencia entre consultas y tablas

Tabla	Consulta que satisface	Clave de partición	Clave de clustering
pasajeros_por_id	Ficha de un pasajero por identificador	pasajero_id	—
aeronaves_por_id	Ficha de una aeronave por identificador	aeronave_id	—
vuelos_por_id	Ficha de un vuelo por identificador	vuelo_id	—
asientos_por_vuelo	Asientos de un vuelo por clase	vuelo_id	clase, numero_asiento
reservas_por_id	Ficha de una reserva por identificador	reserva_id	—
pagos_por_reserva	Pago asociado a una reserva	reserva_id	—
disponibilidad_por_clase	Q1 — conteo ocupados/disponibles	vuelo_id	clase
capacidad_por_clase	Q1 — capacidad por clase	vuelo_id	clase
reservas_por_pasajero	Q2 — historial cronológico	pasajero_id	fecha_reserva DESC, reserva_id
manifiesto_por_vuelo	Q3 — manifiesto enriquecido	vuelo_id	numero_asiento ASC
reservas_por_ruta_mes	Q4 — confirmadas por ruta y mes	origen, destino, anio_mes	fecha_salida, vuelo_id
vuelos_por_ruta_mes	Q4 — capacidad por ruta y mes	origen, destino, anio_mes	fecha_salida, vuelo_id
ingresos_por_vuelo	Q5 — acumulador de ingresos	anio_mes	vuelo_id
top_ingresos_por_mes	Q5 — ranking por ingresos	anio_mes	monto_total DESC, vuelo_id
reservas_pendientes_por_vuelo	Expiración automática por TTL	vuelo_id	reserva_id

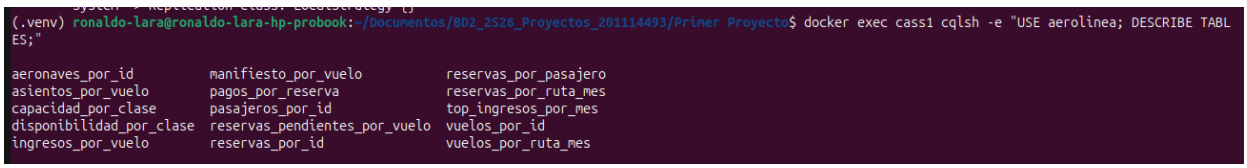


Figura 4. Las quince tablas del keyspace aerolinea.

Los dos esquemas siguientes representan gráficamente las tablas especializadas agrupadas por la consulta que satisfacen, indicando para cada una su clave de partición y su clave de clustering.

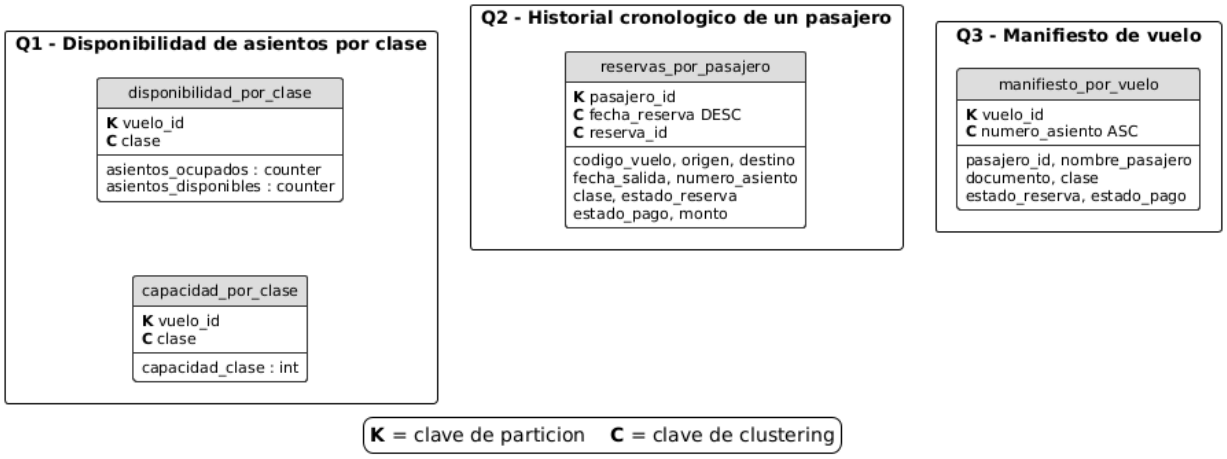


Figura 5. Esquema lógico de las tablas que resuelven las consultas Q1, Q2 y Q3.

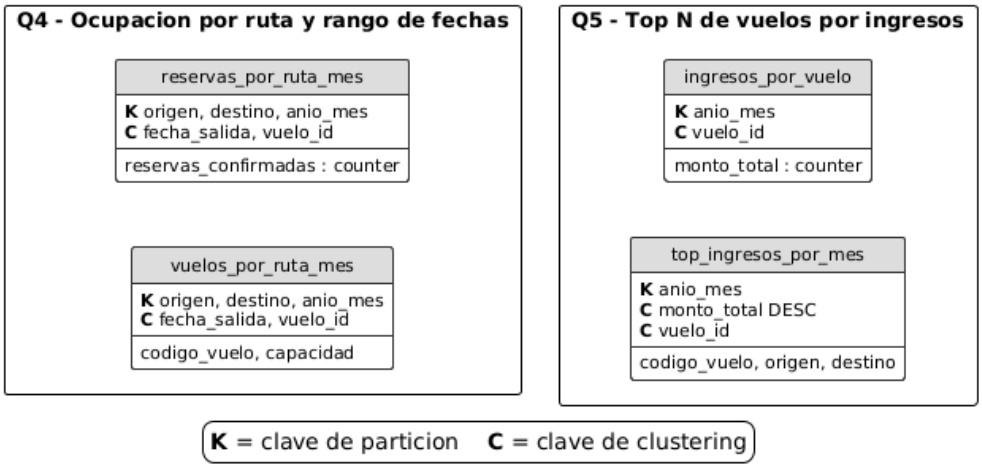


Figura 6. Esquema lógico de las tablas que resuelven las consultas Q4 y Q5.

4.3 Justificación de claves por consulta

Q1 — Disponibilidad de asientos por clase.

Obtener el número de asientos ocupados y disponibles de un vuelo desglosado por clase mediante un conteo de filas exigiría recorrer la partición completa en cada consulta. En su lugar se mantiene la tabla disponibilidad_por_clase con dos columnas de tipo counter que se actualizan en cada reserva y en cada cancelación, de modo que la lectura se reduce a recuperar tres filas ya calculadas. La clave de partición es vuelo_id y la de clustering es clase, lo que agrupa las tres clases del vuelo en una sola partición contigua.

Una tabla que contiene columnas counter no admite columnas de otro tipo fuera de la clave primaria. Esa restricción obliga a almacenar la capacidad nominal de cada clase —un valor entero — en la tabla separada capacidad_por_clase, definida con la misma clave primaria. La consulta completa se resuelve leyendo ambas particiones, ambas residentes en el mismo nodo por compartir la clave de partición.

Q2 — Historial cronológico de un pasajero.

La consulta requiere las reservas de un pasajero dentro de un rango de fechas, ordenadas cronológicamente e incluyendo el estado de pago. La clave de partición es pasajero_id, de modo que todo el historial del pasajero reside en un solo nodo. La primera clave de clustering es fecha_reserva con orden descendente, lo que permite aplicar los comparadores de rango >= y <= directamente sobre el disco: Cassandra localiza el inicio del rango y lee de forma secuencial hasta el final, sin filtrado posterior. Se añade reserva_id como segunda clave de clustering para garantizar unicidad cuando dos reservas del mismo pasajero comparten instante.

La tabla denormaliza atributos de tres entidades: de la reserva conserva el asiento y el estado; del vuelo, el código, el origen, el destino y la fecha de salida; del pago, el estado y el monto. Sin esa copia sería necesario consultar vuelos_por_id y pagos_por_reserva por cada fila devuelta.

Q3 — Manifiesto de vuelo ordenado y enriquecido.

El manifiesto exige el listado de pasajeros de un vuelo ordenado por número de asiento, con clase, estado de reserva y estado de pago. La clave de partición vuelo_id concentra el manifiesto completo en una partición y la clave de clustering numero_asiento en orden ascendente entrega las filas ya ordenadas, sin ordenamiento en el cliente.

El número de asiento se almacena con el formato de tres dígitos y una letra —001A, 042D— en lugar de 1A o 42D. El ordenamiento de una clave de clustering de tipo text es lexicográfico, de modo que sin el relleno de ceros la fila 10A precedería a la 2A. El relleno hace coincidir el orden lexicográfico con el orden numérico esperado.

La tabla incorpora el nombre y el documento del pasajero, que pertenecen a la entidad Pasajeros, y el estado del pago, que pertenece a la entidad Pagos. En un modelo relacional la consulta requeriría dos operaciones JOIN; aquí el costo se paga una sola vez en la escritura.

Q4 — Porcentaje de ocupación por ruta y rango de fechas.

Si la clave de partición fuese únicamente el par origen-destino, una ruta con varios años de operación acumularía todos sus vuelos en una sola partición alojada en un solo nodo. Las particiones de gran tamaño degradan la compactación, incrementan la presión sobre el heap y concentran la carga de lectura, situación conocida como hotspot. Para evitarlo se aplicó bucketing incorporando el periodo mensual a la clave de partición, que queda definida como el trío (origen,

destino, anio_mes). Cada mes de cada ruta constituye así una partición independiente, distribuida por hash entre los tres nodos.

El contrapeso del bucketing es que una consulta sobre un rango que abarca varios meses requiere una lectura por bucket, cuyos resultados se combinan en el cliente. Dado que el tamaño de cada partición queda acotado, el costo es predecible.

Al igual que en Q1, la restricción de las tablas counter obliga a separar el contador de reservas confirmadas —en reservas_por_ruta_mes— de la capacidad de la aeronave —en vuelos_por_ruta_mes—. El porcentaje se obtiene dividiendo ambos valores para cada vuelo.

Q5 — Top N de vuelos por ingresos generados.

La suma de los pagos confirmados se acumula en la tabla ingresos_por_vuelo mediante una columna counter, con anio_mes como clave de partición. Los montos se almacenan en centavos como enteros, ya que el tipo counter no admite valores decimales y la aritmética de punto flotante introduciría error acumulado sobre cien mil operaciones.

El ranking se materializa en top_ingresos_por_mes, donde monto_total figura como primera clave de clustering en orden descendente. Con esa definición las filas quedan físicamente ordenadas de mayor a menor ingreso dentro de la partición del mes, y la cláusula LIMIT N se resuelve leyendo las primeras N filas, sin ordenamiento ni filtrado en el cliente.

Incluir el monto en la clave de clustering tiene una consecuencia operativa: el valor no puede actualizarse en el sitio, porque forma parte de la identidad de la fila. Cuando el ingreso de un vuelo cambia debe eliminarse la fila anterior e insertarse la nueva. En el presente proyecto la tabla se construye una vez finalizada la carga, a partir de los contadores ya consolidados.

5. Configuración de TTL

El enunciado exige la eliminación automática de reservas expiradas. La tabla reservas_pendientes_por_vuelo se definió con un tiempo de vida predeterminado de 86,400 segundos, equivalente a veinticuatro horas:

```
CREATE TABLE reservas_pendientes_por_vuelo (
    vuelo_id uuid,
    reserva_id uuid,
    pasajero_id uuid,
    numero_asiento text,
    fecha_reserva timestamp,
    PRIMARY KEY ((vuelo_id), reserva_id)
) WITH default_time_to_live = 86400;
```

Transcurrido ese plazo, Cassandra marca las filas con una lápida (tombstone) y las elimina definitivamente durante la siguiente compactación. Las reservas en estado pendiente se registran en esta tabla durante la carga; las confirmadas no, puesto que no deben expirar. Cabe señalar que las tablas con columnas counter no admiten TTL, razón adicional por la que el mecanismo se aplica sobre una tabla de estado y no sobre los contadores.

6. Carga masiva de datos

La generación e inserción de datos se implementó en Python con el driver oficial cassandra-driver. El procedimiento recorre 520 vuelos y, para cada uno, genera sus asientos y las reservas correspondientes, escribiendo simultáneamente en todas las tablas que dependen de esos datos.

Concepto	Valor
Pasajeros generados	25,000
Aeronaves generadas	40
Vuelos generados	520
Reservas insertadas	100,080
Tiempo total de carga	105.1 segundos
Tamaño de batch	25 sentencias
Carga por nodo tras la inserción	32.74 MiB (distribución uniforme)

Se emplearon Batch Writes de tipo UNLOGGED con veinticinco sentencias por lote. La elección del tamaño responde a un compromiso: los lotes reducen el número de viajes de red, pero un lote demasiado grande —o que abarque muchas particiones distintas— sobrecarga al nodo coordinador, que debe distribuir cada sentencia a su réplica correspondiente. Durante las pruebas iniciales, lotes de cincuenta sentencias con doscientas operaciones asíncronas simultáneas provocaron errores de tipo WriteTimeout; la configuración final mantiene un máximo de ocho lotes en vuelo.

Las actualizaciones de columnas counter se agrupan en lotes de tipo COUNTER, separados de los lotes de inserción ordinaria, ya que Cassandra no permite combinar ambos tipos en una misma operación. Los contadores constituyen la operación de escritura más costosa del proceso, puesto que internamente requieren una lectura previa a la escritura.

La generación respeta las reglas de negocio: los asientos de cada vuelo se seleccionan por muestreo sin reposición, lo que garantiza que un asiento se asigne a un único pasajero y que el número de reservas nunca supere la capacidad de la aeronave; los vuelos cancelados propagan el estado cancelada y reembolsado a todas sus reservas y pagos; y toda reserva genera un pago con el estado correspondiente.

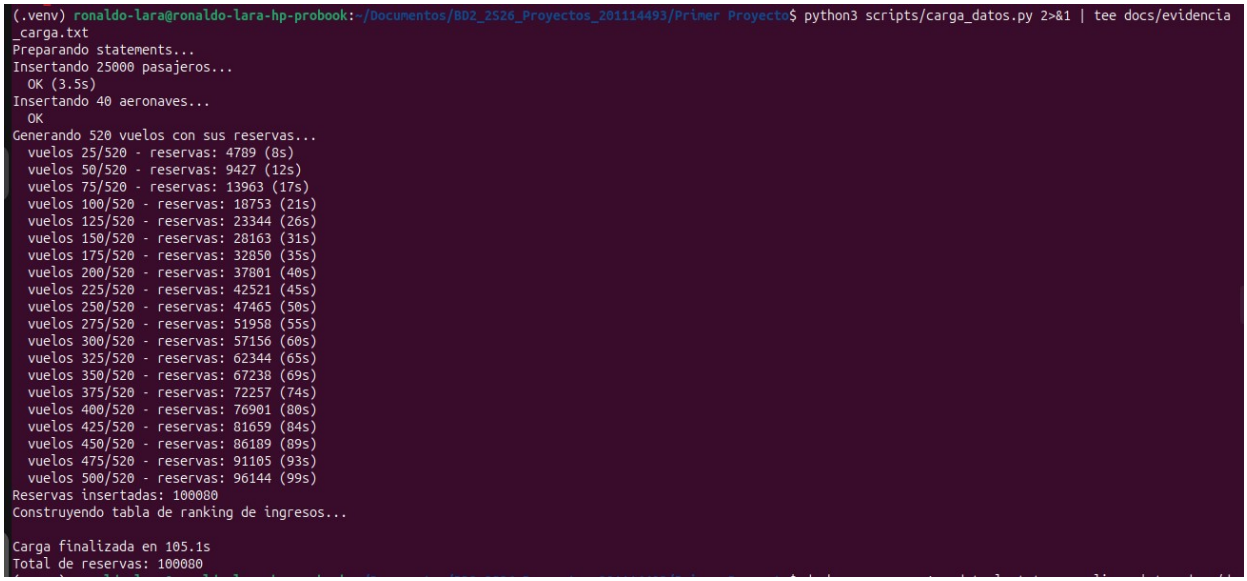


Figura 7. Finalización de la carga masiva: 100,080 reservas insertadas en 105.1 segundos.

7. Consultas implementadas y rendimiento

Las cinco consultas se ejecutaron con nivel de consistencia QUORUM sobre el conjunto de datos completo. Cada medición corresponde al promedio de veinte ejecuciones, descartando una ejecución previa de calentamiento:

Consulta	Estrategia de modelado	Filas	Prom. (ms)	Mín. (ms)
Q1 Disponibilidad por clase	Contadores (counter)	3	8.70	2.73
Q2 Historial de pasajero	Rango sobre clustering key	5	54.54	4.34
Q3 Manifiesto de vuelo	Denormalización, una partición	164	27.05	7.15
Q4a Confirmadas por ruta-mes	Bucketing + contador	2	3.95	2.78
Q4b Capacidad por ruta-mes	Bucketing	2	5.32	3.28
Q5 Top 10 por ingresos	Clustering DESC + LIMIT	10	6.20	2.73

Ninguna de las consultas requiere ALLOW FILTERING. Los tiempos mínimos, comprendidos entre 2.73 y 7.15 milisegundos, reflejan el costo real del acceso a una partición única; los promedios superiores y la dispersión observada en Q2 y Q3 se explican por la contención de memoria del entorno de pruebas, que dispone de ocho gigabytes compartidos entre los tres nodos y el sistema operativo, y no por el diseño del modelo.

El seguimiento de ejecución obtenido con TRACING ON sobre la consulta Q3 confirma el comportamiento esperado. La traza registra la actividad «Executing single-partition query on manifiesto_por_vuelo», seguida de la adquisición de referencias a una sola SSTable y un acierto de caché de claves. No se registra recorrido del anillo ni descarte de filas por filtrado.

```
Execute CQL3 query                                              0 us
Parsing SELECT ... FROM manifiesto_por_vuelo WHERE ...        191 us
Preparing statement                                           313 us
Executing single-partition query on manifiesto_por_vuelo   136277 us
Acquiring sstable references                               136626 us
Skipped 0/1 non-slice-intersecting sstables                136845 us
Key cache hit for sstable 15                               137020 us
Merged data from memtables and 1 sstables                  138610 us
Read 64 live rows and 0 tombstone cells                    138969 us
Request complete                                           139870 us
```

El resultado funcional de Q4 sobre la ruta PTY–LIM en julio de 2026 ilustra la resolución completa del porcentaje de ocupación combinando contador y capacidad: el vuelo AV663 del 23 de julio registra 144 reservas confirmadas sobre 240 asientos, equivalente a 60.0 por ciento de ocupación; el vuelo AV313 del 27 de julio registra 131 sobre 240, equivalente a 54.6 por ciento.

8. Pruebas de tolerancia a fallos

8.1 Procedimiento

Se diseñó una prueba comparativa sobre la consulta Q3, ejecutada treinta veces por cada nivel de consistencia en tres escenarios sucesivos: clúster íntegro, clúster con el nodo cass3 detenido y clúster restablecido tras la reincorporación del nodo y la ejecución de nodetool repair. Se registró el número de ejecuciones exitosas y fallidas junto con la latencia promedio, mínima y el percentil 95.

8.2 Resultados

Nivel	3 nodos activos	Nodo cass3 detenido	Clúster restablecido
ONE	30/30 — 11.79 ms	30/30 — 3.50 ms	30/30 — 7.89 ms
QUORUM	30/30 — 11.51 ms	30/30 — 6.08 ms	30/30 — 7.86 ms
ALL	30/30 — 15.81 ms	0/30 — Unavailable	30/30 — 8.14 ms

```
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ python3 scripts/pruebas_fallos.py "3 nodos activos" 2>&1
 | tee docs/fallos_1_completo.txt

======================================================================
ESCENARIO: 3 nodos activos
Consulta: manifiesto del vuelo AV369
Repeticiones por nivel: 30
======================================================================
CL        exitosas  fallidas   prom(ms)   min(ms)   p95(ms)  error
ONE             30         0      11.79      3.29     62.46
QUORUM          30         0      11.51      4.98     35.38
ALL             30         0      15.81      5.51     44.66
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ docker stop cass3
cass3
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ sleep 40
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ docker exec cass1 nodetool status | tee docs/fallos_2_st
atus_caido.txt
Datacenter: dc1
==============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Host ID                               Rack
UN  172.19.0.2  32.81 MiB  16      100.0%            5af76af3-732e-4761-8ef4-8100a3b599bf  rack1
DN  172.19.0.4  32.74 MiB  16      100.0%            4ce13d2a-99f8-4b45-b0bc-262bee75ff3b  rack1
UN  172.19.0.3  32.81 MiB  16      100.0%            4873727c-c1bc-48d3-8980-b3fa22a64e75  rack1
```

Figura 8. Escenario 1: clúster íntegro. Los tres niveles completan las treinta ejecuciones.

```
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ python3 scripts/pruebas_fallos.py "nodo cass3 caido" 2>&
1 | tee docs/fallos_2_caido.txt

======================================================================
ESCENARIO: nodo cass3 caido
Consulta: manifiesto del vuelo AV369
Repeticiones por nivel: 30
======================================================================
CL        exitosas  fallidas   prom(ms)   min(ms)   p95(ms)  error
ONE             30         0       3.50      3.03      4.26
QUORUM          30         0       6.08      5.11      6.69
ALL              0        30          -         -         -  Unavailable
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ docker start cass3
cass3
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ sleep 90
(.venv) ronaldo-lara@ronaldo-lara-hp-probook:~/Documentos/BD2_2S26_Proyectos_201114493/Primer Proyecto$ docker exec cass1 nodetool status | tee docs/fallos_3_st
atus_recuperado.txt
Datacenter: dc1
==============
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Host ID                               Rack
UN  172.19.0.2  32.81 MiB  16      100.0%            5af76af3-732e-4761-8ef4-8100a3b599bf  rack1
UN  172.19.0.4  37.41 MiB  16      100.0%            4ce13d2a-99f8-4b45-b0bc-262bee75ff3b  rack1
UN  172.19.0.3  32.81 MiB  16      100.0%            4873727c-c1bc-48d3-8980-b3fa22a64e75  rack1
```

Figura 9. Escenario 2: con el nodo 172.19.0.4 en estado DN, el nivel ALL falla con Unavailable.

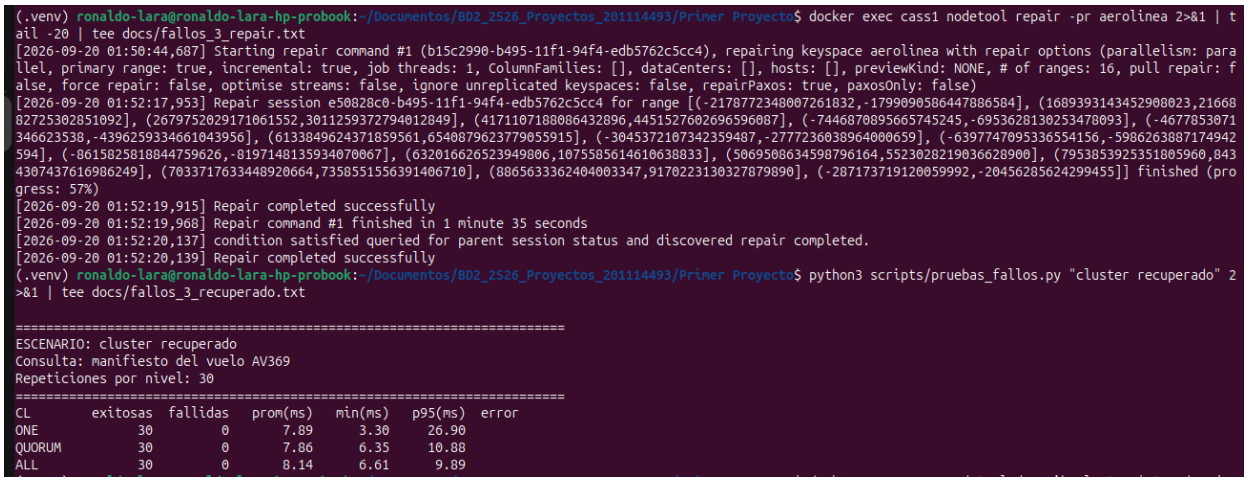


Figura 10. Escenario 3: reparación completada y restablecimiento del nivel ALL.

8.3 Análisis

Con el clúster íntegro los tres niveles responden a la totalidad de las peticiones, y la latencia crece de forma monótona conforme aumenta el número de réplicas exigidas: 11.79 milisegundos con ONE, 11.51 con QUORUM y 15.81 con ALL. El coordinador debe esperar a la réplica más lenta, de modo que ALL queda expuesto a la variabilidad de cualquiera de los tres nodos.

Con el nodo cass3 detenido, el nivel ALL falla en la totalidad de las treinta ejecuciones con la excepción Unavailable. El comportamiento es el esperado: con factor de replicación 3, ALL exige la confirmación de las tres réplicas y una de ellas no está disponible. Los niveles ONE y QUORUM mantienen el cien por ciento de disponibilidad, puesto que requieren una y dos réplicas respectivamente, ambas alcanzables con dos nodos activos. Este contraste constituye la demostración práctica del compromiso entre consistencia y disponibilidad que plantea el teorema CAP.

Un resultado aparentemente contraintuitivo merece comentario: con un nodo menos, ONE y QUORUM registraron latencias inferiores a las del escenario íntegro —3.50 y 6.08 milisegundos frente a 11.79 y 11.51—. La explicación no está en el protocolo sino en el entorno: al detener un contenedor se liberaron aproximadamente 512 megabytes de heap y la presión sobre la memoria del anfitrión disminuyó, reduciendo la paginación. La medición ilustra que, en entornos de recursos limitados, la contención local puede dominar sobre el costo del protocolo de consistencia.

Tras reiniciar el contenedor, el nodo se reincorporó al anillo en menos de noventa segundos y la operación nodetool repair completó la sincronización en un minuto y treinta y cinco segundos, procesando los dieciséis rangos de token primarios del nodo. La carga del nodo restablecido quedó en 37.41 MiB frente a 32.81 MiB de los otros dos; la diferencia corresponde a SSTables recibidas durante la transmisión de la reparación que aún no habían sido compactadas, y no a datos duplicados.

Conviene precisar el alcance de la reparación en este escenario. Durante la indisponibilidad del nodo no se ejecutaron operaciones de escritura, por lo que no existían divergencias reales entre réplicas. La ejecución de nodetool repair se realizó como procedimiento de verificación explícita de la recuperación. En un entorno con escrituras concurrentes, el mecanismo de hinted handoff habría retenido las mutaciones dirigidas al nodo ausente durante una ventana de tres horas, y la

reparación sería el único procedimiento capaz de garantizar la convergencia completa más allá de esa ventana.

9. Conclusiones

El modelado dirigido por consultas produjo un esquema de quince tablas físicas a partir de seis entidades conceptuales. La multiplicación no responde a redundancia accidental sino a una decisión de diseño: cada patrón de acceso dispone de una estructura propia, de modo que toda consulta del negocio se resuelve en una única partición y sin operaciones de unión. El costo se traslada a la escritura, donde una sola reserva se materializa en cinco tablas distintas.

Las mediciones confirman la eficacia del enfoque. Las cinco consultas se ejecutan por debajo de los ocho milisegundos en su mejor caso sobre un conjunto de cien mil reservas, sin recurrir a ALLOW FILTERING. Las estrategias específicas —contadores para las agregaciones de ocupación e ingresos, bucketing temporal para acotar el tamaño de las particiones de ruta, y ordenamiento físico por clave de clustering para resolver el ranking— resultaron determinantes: sin ellas, las consultas Q1, Q4 y Q5 habrían requerido agregación en el cliente o recorrido completo del anillo.

Las pruebas de tolerancia a fallos cuantificaron el compromiso entre consistencia y disponibilidad. Con factor de replicación 3, el nivel QUORUM constituye el punto de equilibrio: conserva consistencia fuerte por cumplir $R + W > RF$ y mantiene disponibilidad plena ante la pérdida de un nodo, condición que ALL no satisface. La elección del nivel de consistencia es, por tanto, una decisión por operación y no una configuración global del sistema.

Finalmente, la restricción de las tablas de tipo counter —que no admiten columnas ordinarias ni TTL— condicionó el diseño más de lo previsto, obligando a desdoblar en dos tablas tanto la consulta de disponibilidad como la de ocupación por ruta. Es un ejemplo de cómo, en Cassandra, las limitaciones del motor se trasladan directamente a la estructura del modelo lógico.

10. Referencias

Apache Software Foundation. Apache Cassandra Documentation, versión 4.1. https://cassandra.apache.org/doc/4.1/

DataStax. Data Modeling for Apache Cassandra. DataStax Academy.

DataStax. Python Driver for Apache Cassandra, documentación oficial. https://docs.datastax.com/en/developer/python-driver/